Расширенный курс языка C++

К.А. Кутасин, Л. М. Львова

Домашнее задание 2

Требования к программам

1. Для решения каждой задачи предлагается реализовать класс на языке программирования C++.
2. Класс должен называться в точности как указано в задании.
3. Класс должен реализовывать все методы указанные в задании.
4. Исходный код класса должен содержаться в двух файлах: **classname.h**, **classname.cpp**, где **classname** – указанное в задании имя класса.
5. Исходный код должен быть оформлен согласно набору правил и рекомендаций стандарта кодирования GNU.
6. Задания включают в себя работу с объектом **some_object**. Содержание этого объекта может быть произвольным, однако гарантируется, что у него есть конструктор по умолчанию, конструкторы копирования и перемещения, а так же copy-assignment и move-assignment операторы.
7. Использовать реализации аналогичных классов из стандартной библиотеки языка в качестве подлежащей реализации запрещено.
8. В случае возникновения вопросов по условию, можно использовать сайт **cppreference.com**. Поведение класса, описанное на этом сайте считается верным.
9. Использовать примеры реализации класса из интернета, в частности, с сайта **cppreference.com** запрещено. Использование искусственного интеллекта запрещено.
10. Проверка задания осуществляется путем использования реализованного класса в тестовой программе. Тестовая программа использует все указанные в задании методы и никакие другие.
11. Если тестовая программа компилируется и корректно работает с приведенной реализацией класса, а реализация соответствует приведенным требованиям, то задание считается выполненным.
12. Если тестовая программа не компилируется или некорректно работает с приведенной реализацией класса, то задание считается не выполненным.
13. Если реализация класса не соответствует приведенным требованиям, то задание считается не выполненным.

Задачи

1. Написать класс **unique_pointer_object**, реализующий единоличное владение указателем на объект с именем **some_object**. Конструктор класса принимает указатель на объект (созданный внешним образом оператором **new**), деструктор класса освобождает память по этому указателю (с помощью оператора **delete**). Класс должен поддерживать конструктор по умолчанию. Класс должен поддерживать конструктор перемещения и move-assignment оператор. Конструктор копирования, а так же copy-assignment оператор должны быть удалены. Класс должен возвращать неконстантный указатель на объект неконстантным методом **get ()**.

2. Написать класс **unique_pointer_array**, реализующий единоличное владение указателем на динамический массив целых чисел (типа **int**). Конструктор класса принимает указатель на массив (созданный внешним образом оператором **new []**), деструктор класса освобождает память по этому указателю (с помощью оператора **delete []**). Класс должен поддерживать конструктор по умолчанию. Класс должен поддерживать конструктор перемещения и move-assignment оператор.

Конструктор копирования, а так же copy-assignment оператор должны быть удалены. Класс должен возвращать неконстантный указатель на массив неконстантным методом **get ()**.

3. Написать класс **flexible_pointer**, реализующий хранение указателя на объект с именем **some_object**. Конструктор класса принимает указатель на объект (созданный внешним образом оператором **new**), а так же флаг владения (типа **bool**). Деструктор класса освобождает память по этому указателю только если флаг владения соответствует значению **true**. Класс должен поддерживать конструктор по умолчанию. Класс должен поддерживать конструктор перемещения и move-assignment оператор. Конструктор копирования, а так же copy-assignment оператор должны быть удалены. Класс должен возвращать неконстантный указатель на объект неконстантным методом **get ()**. Класс должен возвращать флаг владения константным методом **is_owner ()**.

4. Написать класс **shared_pointer**, реализующий разделенное владение указателем на объект с именем **some_object**. Конструктор класса принимает указатель на объект (созданный внешним образом оператором **new**). Деструктор класса освобождает память по этому указателю только если у класса не осталось копий. Класс должен поддерживать конструктор по умолчанию. Класс должен поддерживать конструктор копирования, конструктор перемещения, а так же copy-assignment и move-assignment операторы. Класс должен возвращать неконстантный указатель на объект неконстантным методом **get ()**. **Копирование класса должно выполнять копирование хранящегося в нем указателя на объект, а не самого объекта.**

5. Написать класс **copiable_unique_pointer**, реализующий единоличное владение указателем на объект с именем **some_object**. Конструктор класса принимает указатель на объект (созданный внешним образом оператором **new**). Деструктор класса освобождает память по этому указателю. Класс должен поддерживать конструктор по умолчанию. Класс должен поддерживать конструктор копирования, конструктор перемещения, а так же copy-assignment и move-assignment операторы. Класс должен возвращать неконстантный указатель на объект неконстантным методом **get ()**. **Копирование класса должно выполнять копирование хранящегося в нем объекта, а не указателя на объект.**

6. Написать класс **elapsed_warning**, который подсчитывает собственное время жизни. Конструктор класса принимает на вход целое число, задающее время в секундах. Деструктор класса должен вывести в поток вывода **stderr** число 1, в случае если время жизни класса превысило заданное в конструкторе время. Класс должен поддерживать конструктор по умолчанию. Класс должен поддерживать конструктор перемещения и move-assignment оператор. Конструктор копирования, а так же copy-assignment оператор должны быть удалены. Класс должен возвращать свое текущее время жизни в секундах (округленное вниз) константным методом **get_lifetime ()**.